

Praneeth (07 april)

Gemma 4 Inference Capacity & Concurrency

Gemma 4 26B-A4B-it model on a single NVIDIA A10G GPU (24,576 MB VRAM).

The Input Context Size for our current deployment is capped at **8,192** tokens.

1. Hardware Resource Allocation (VRAM Budget)

To ensure low-latency inference, we have allocated 95% of the available VRAM to the vLLM engine. The physical memory is partitioned as follows:

Component	VRAM Usage (Approx)	Description
Model Weights	16,800 MiB	26B parameters compressed to 4-bit AWQ.
CUDA & CUDAGraphs	~1,300 MiB	Memory used for the GPU's "fast-track" execution blueprints.
KV Cache Buffer	5,502 MiB	The memory pool dedicated to conversation history/context.
Safety Buffer	1,228 MiB	Reserved for temporary CUDA spikes and allocations.
Total A10G VRAM	24,576 MiB	

2. The KV Cache Calculus (Token Cost)

The KV Cache is the memory that stores the "Key" and "Value" tensors for every previous word in a conversation. Gemma 4's architecture is significantly more "memory-expensive" per token than smaller models.

Architectural Parameters:

- Layers: 48
- KV Heads: 8
- Average Head Dimension: 256
- Precision (FP16): 2 bytes

Formula for 1 Token:

$\text{Memory} = (\text{Layers} * \text{Heads} * \text{Head_Dim} * \text{Precision}) * 2 \text{ (K and V)}$
 $48 * 8 * 256 * 2 * 2 = 393,216 \text{ Bytes } (\sim 384 \text{ KB})$

Based on the current 5.5GB KV Cache allocation, the server has a hard ceiling of **14,752** Total Tokens.

3. Concurrency Projections

Concurrency is calculated by dividing the total token pool (14,752) by the sum of the System Prompt + Conversation History + Target Generation Length per user.

According to our discussions : (**For english**).

- Average context per user: 3,000 Tokens
- Math: $14,752 / 3,000$
- Projected Concurrency: **~4-5 Simultaneous Users**

FOR **HINDI** IT VARIES :

It might come around **2-3 Simultaneous Users**

4. Key Performance Insights

- MoE Efficiency: While the memory footprint remains constant at 26B, the compute load only taxes the GPU for 4B active parameters per token. This allows for high throughput (tokens/second) even if concurrency is limited by VRAM.

- Prefix Caching: By enabling `--enable-prefix-caching`, the System Prompt (Rules) only consumes memory once for the entire server. This saves ~1,500 tokens of memory *per user* if all users are sharing the same script.
- CUDAGraph Optimization: We have accepted a small VRAM "tax" (~1GB) to enable CUDAGraphs, which significantly reduces Time To First Token (TTFT) by eliminating CPU dispatch overhead.